Advanced Cloud Computing · Lab IA

CarbonShift

An ML-based cloud resource manager that predicts workload and grid carbon intensity and allocates resources on AWS

Name	Roll No.
Tanish Gupta	16010123132
Tanish Shah	16010123133
Tanisha Mukherjee	16010123134

Semester VII · Submitted 18 October 2026

Code: github.com/StarkFurry64/Carbonshift (private)

Abstract

Cloud batch jobs (reports, model training, data processing) often do not need to run the moment they are submitted. **CarbonShift** exploits that flexibility: it forecasts the carbon intensity of the electricity grid and the incoming workload 24 hours ahead with **XGBoost**, and a rolling-horizon scheduler decides **when**, **where** and **on how many virtual machines** each job runs so that every job meets its deadline with the least CO_2. Using four years of hourly Electricity Maps data for the Western (IN-WE) and Southern (IN-SO) Indian grids, the forecaster reduces the error of the strongest simple baseline by 19% and 13% respectively. In a full-year simulation of 70,189 jobs, CarbonShift emits **31% less CO_2** than running jobs immediately, stays within **1.1%** of an oracle with perfect knowledge of the future and misses **0.00%** of deadlines. The system was deployed on **AWS** (EC2, SQS, DynamoDB): a laptop controller drives four t3.micro workers in real time, and on identical job streams CarbonShift used **24% less CO_2** than the FIFO baseline on the real VMs. We also evaluate what happens when forecasts are wrong (surprise cloudy days, workload bursts) and show how a confidence gate and safety buffer protect deadlines.

Keywords: carbon-aware computing, workload forecasting, XGBoost, autoscaling, AWS EC2, SQS, DynamoDB, temporal and spatial load shifting.

Contents

1. Introduction..... 2
2. System architecture..... 2
3. Input data and features..... 3
4. Machine-learning model and prediction horizon..... 5
5. Resource allocation and autoscaling design..... 6
6. Implementation on AWS..... 7
7. Evaluation: full-year simulation..... 9
8. Evaluation: live runs on AWS..... 10
9. What happens when the prediction is wrong..... 12
10. Discussion and limitations..... 14
11. Conclusion and future work..... 14
References..... 14
Appendix A. How to reproduce..... 14

1. Introduction

Data centres consume roughly 1–1.5% of global electricity, and the CO_2 emitted per kilowatt-hour depends strongly on **where** and **when** that electricity is produced. In India, coal still dominates at night, while solar generation lowers the carbon intensity of the grid during the day; the Southern grid also has more wind and hydro than the Western grid. Batch workloads are an ideal lever: a job with a 12-hour deadline can be moved to the cleanest hour, or to the cleaner region, without the user noticing.

The assignment asks for an **ML-based cloud resource manager that predicts workload and allocates resources**. CarbonShift does this with two forecasters and a scheduler, and allocates real AWS virtual machines accordingly. Table 1 maps the five assignment requirements to the sections of this report.

Table 1. Assignment requirements and where they are covered.

Assignment requirement	Where it is answered
1. Identify input data and features	Section 3 (data, cleaning, EDA, features)
2. Choose the ML algorithm and prediction horizon	Section 4 (baselines vs XGBoost, 1–24 h ahead)
3. Design the resource allocation / autoscaling mechanism	Section 5 (scheduler, VM controller) and Section 6 (AWS)
4. Evaluate prediction accuracy, utilisation, latency, cost and energy	Sections 4.4, 7 and 8
5. Discuss what happens when the prediction is wrong	Section 9 (failure injection, gate, buffer)

1.1 Objectives and contributions

- A cleaned, four-year hourly carbon-intensity dataset for two Indian grid zones, with exploratory analysis of its daily and seasonal patterns.
- Direct multi-horizon XGBoost forecasters (1–24 h) for grid carbon intensity and for incoming batch workload, compared with simple baselines.
- A rolling-horizon, deadline-aware scheduler with a confidence gate and safety buffer, evaluated against FIFO, a simple carbon rule and an oracle.
- A real AWS deployment (EC2, SQS, DynamoDB, IAM) driven by a laptop controller that autoscales VMs ahead of planned work, plus a live Streamlit dashboard.
- A failure analysis that injects wrong forecasts and workload bursts, and an honest comparison of simulated and measured results.

2. System architecture

Figure 1 shows the system. The **control plane** runs on a laptop: it holds the forecasting models, the scheduler, the VM controller and the dashboard. The **data plane** runs on AWS: one Amazon SQS queue and two t3.micro EC2 workers per grid group, and one DynamoDB table that collects results. Every simulated hour (one real minute in the demo, configurable) the controller lets new jobs arrive, re-plans all waiting jobs against fresh forecasts, sends the jobs scheduled for this hour to the right queue, starts or stops VMs, and reads finished results back.

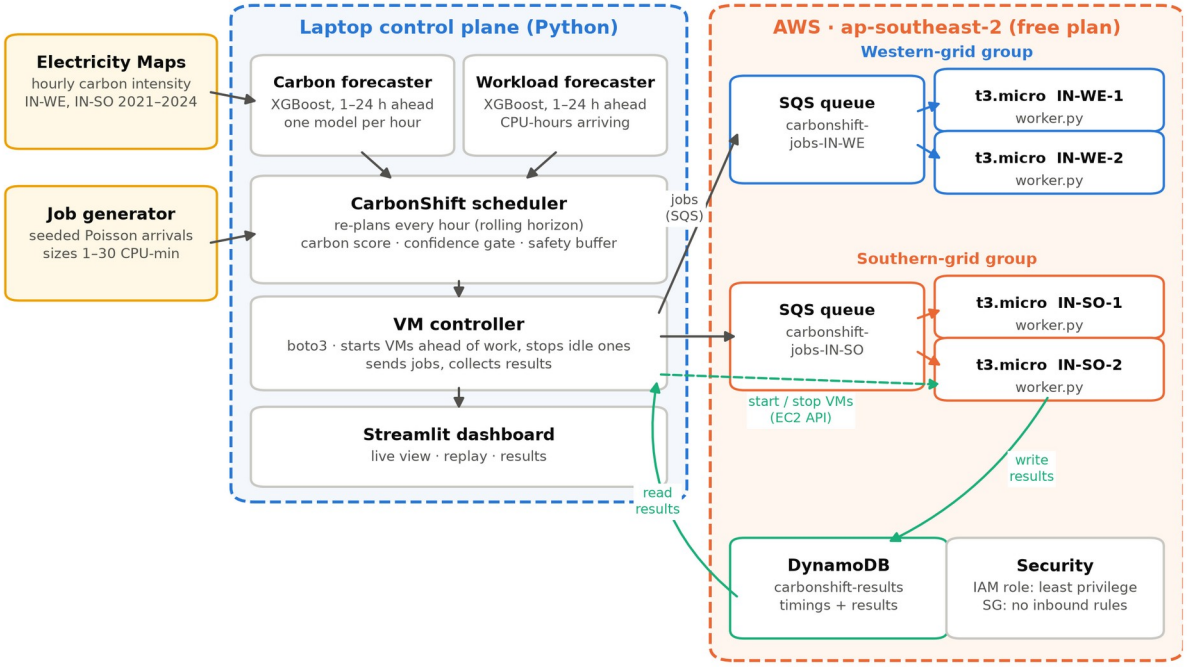


Figure 1. CarbonShift architecture: laptop control plane and AWS data plane.

Deployment constraints. The AWS account is on the free plan ($120 credit), which allows a single region, ap-southeast-2 (Sydney), and has an 8-vCPU on-demand quota. Both worker groups therefore run in Sydney, and each group is bound to one Indian grid zone's carbon trace (IN-WE or IN-SO). Because the live demo replays the 2024 test year, the physical location of the VMs does not change any result; it only affects network latency, which is negligible for batch jobs.

3. Input data and features

3.1 Grid carbon intensity

We use the Electricity Maps hourly datasets for zones **IN-WE** (Western India) and **IN-SO** (Southern India), 2021–2024: 35,064 hours per zone, with no gaps or duplicates. The target is the **life-cycle (LCA) carbon intensity** in gCO_2eq/kWh; the direct intensity is kept as an option. One single-hour spike (41 g against a local median of 468 g, IN-SO, 19 Jan 2023) was replaced by interpolation. About 96% of the values are Electricity Maps model estimates rather than direct measurements, which we list as a limitation. Training uses 2021–2023 and testing 2024, split by time and never shuffled.

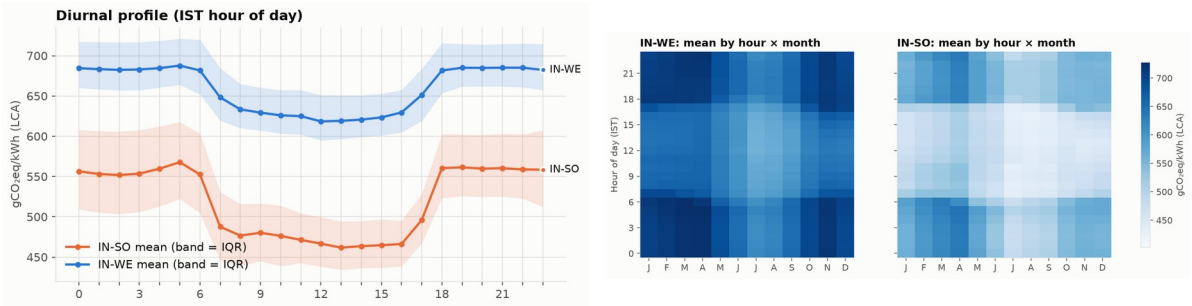


Figure 2. Left: average carbon intensity by hour of day (IST). Right: average by hour and month. The midday solar dip and the cleaner monsoon months are clearly visible.

Table 2. Key patterns found in the exploratory analysis.

Statistic (2021–2024)	IN-WE (Western)	IN-SO (Southern)
Mean LCA intensity (g/kWh)	659	519
Typical within-day range (g/kWh)	77	119
Cleanest / dirtiest hour (IST)	12:00 / 5:00	13:00 / 5:00
Cleanest / dirtiest month	Jul / Apr	Jul / Apr
Weekday vs weekend mean	659 / 659	521 / 517

Three findings drive the design: (i) a strong **24-hour cycle** with a dip between about 07:00 and 17:00 IST, so delaying work towards midday saves carbon; (ii) the Southern grid was **cleaner than the Western grid in 100% of the hours of 2024** (by 117 g/kWh on average), so the region choice matters as much as timing; (iii) there is no weekday/weekend effect, so the day-of-week feature carries little information for carbon.

3.2 Batch workload

The workload is a seeded synthetic job stream: Poisson arrivals with an IST office-hours pattern (1.8× during 09:00–18:00, 0.4× at night, 0.6× at weekends; 8 jobs per hour on average), log-normal job sizes clipped to 1–30 CPU-minutes (median 5, mean 7.2), and a deadline slack of 2, 6, 12 or 24 hours. 2024 contains 70,189 jobs. The same seed produces the same jobs for every policy and slack value, so all comparisons are paired.

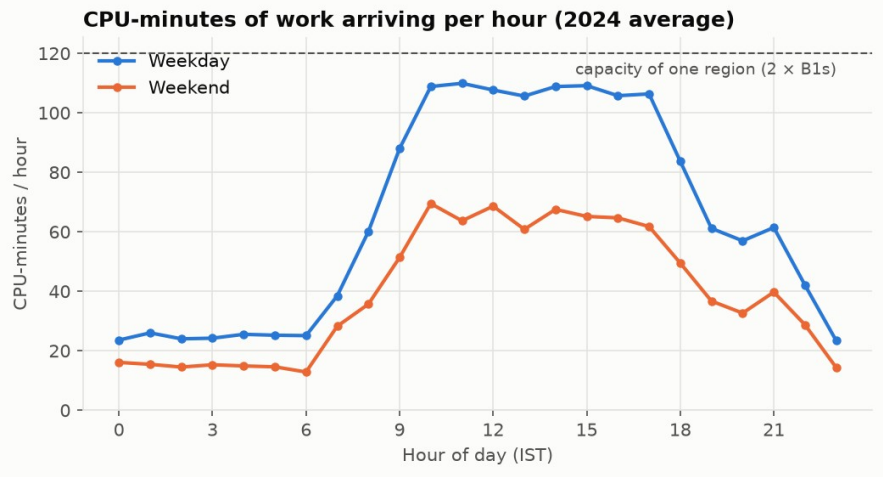


Figure 3. Average CPU-minutes of work arriving per hour. Weekday office-hour peaks approach the capacity of one group (two VMs).

3.3 Features

Table 3. Features used by the two forecasters (all known at forecast time; no leakage).

Feature group	Carbon forecaster	Workload forecaster
Recent values (lags)	t−1, t−2, t−3, t−24, t−168	t−1, t−2, t−3, t−24, t−168
Rolling statistics	24 h mean and std, 24 h change of the mean	24 h and 168 h mean
Seasonal references	same hour yesterday, last week, 7-day same-hour mean	same hour yesterday / last week, 4-week same hour & weekday
Calendar (of target hour)	hour, day of week, month (IST)	hour, day of week (IST)
Other	carbon-free energy %, latest change vs yesterday	–

A unit test verifies that no feature uses future information: changing every value after the forecast origin leaves all features unchanged.

4. Machine-learning model and prediction horizon

4.1 Horizon

Deadlines in our workload are at most 24 hours and the data is hourly, so both forecasters predict **1 to 24 hours ahead in hourly steps**. The scheduler re-plans every hour with fresh forecasts (rolling horizon), so short-horizon accuracy matters most for urgent decisions and long-horizon accuracy for deciding whether to wait.

4.2 Models

We compare three baselines: **persistence** (same hour yesterday), a **24-hour moving average**, and a **7-day same-hour average**, against **XGBoost** (gradient-boosted trees) with one model per horizon (direct multi-horizon strategy, 24 models per zone). Early stopping uses the last 90 days of the training period, then each model is refit on all of 2021–2023.

An important design choice came from a failed first attempt: predicting the change from the latest reading made XGBoost *worse* than persistence at mid horizons, because trees approximate the large, almost linear daily swing poorly. Predicting the **correction to "same hour yesterday"** (the residual over persistence) fixed this: XGBoost then beats every baseline at every horizon.

4.3 Carbon forecast accuracy (test year 2024)

Table 4. Average error over horizons 1–24 h (MAE in gCO_2eq/kWh).

Model	IN-WE MAE	IN-WE MAPE	IN-SO MAE	IN-SO MAPE
Persistence (same hour yesterday)	8.47	1.31%	11.46	2.19%
Moving average (24 h)	27.30	4.18%	47.52	8.91%
Seasonal MA (7 days)	12.93	2.03%	16.05	3.04%
XGBoost (ours)	6.87	1.06%	10.00	1.90%

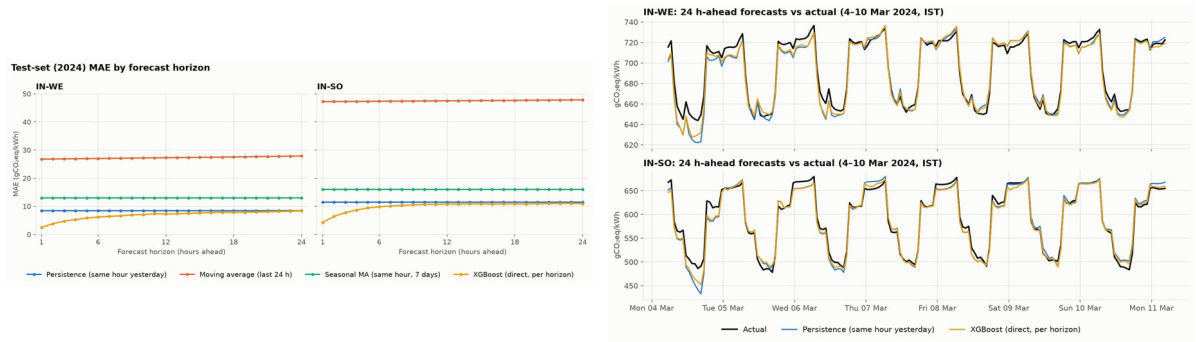


Figure 4. Left: MAE by horizon for both zones (XGBoost in yellow). Right: 24-hour-ahead forecasts against the actual values for one week of 2024.

XGBoost's error grows from 2.5 g at 1 h to 8.4 g at 24 h for IN-WE, and from 4.3 g to 10.9 g for IN-SO, about 1–2% of the signal. It improves on persistence by 19% (IN-WE) and 13% (IN-SO). This error is an order of magnitude smaller than the 70–120 g daily swing, which is why shifting work in time is worthwhile; the per-horizon MAE is also used by the scheduler's confidence gate (Section 5).

4.4 Workload forecast

The workload forecaster predicts CPU-hours arriving per hour, again 1–24 h ahead. It is trained on a separate 2021–2023 stream and tested on the 2024 stream used everywhere else.

Table 5. Workload forecast error (mean arrival rate 57.6 CPU-min/h).

Model	MAE (CPU-min/h)	RMSE (CPU-min/h)
Persistence (same hour yesterday)	32.0	43.1
Same hour last week	29.6	40.1
4-week same hour & weekday	23.7	31.4
XGBoost (ours)	21.4	28.5

Because arrivals are random, much of the remaining error is irreducible. The scheduler therefore uses the workload forecast conservatively: it keeps 50% of the forecast arrivals free in future hours, so deferred jobs do not crowd out work that has not arrived yet.

5. Resource allocation and autoscaling design

5.1 Rolling-horizon scheduler

Every hour the scheduler re-plans **all** waiting jobs against the latest 24-hour forecasts and commits only the jobs placed in the current hour; the rest are re-planned an hour later. Per job, in earliest-deadline-first order:

1. **Urgent jobs run now.** If no feasible hour remains outside the safety buffer (1 h), the job runs immediately in the group with the lowest forecast carbon intensity that has capacity.
2. **Score every feasible (group, hour) inside the deadline:** score = α · CI_forecast · (job energy + idle energy of any extra VM it would switch on) + β · extra VM cost + γ · delay. With α = 1, β = 0, γ = 0.001 the scheduler is carbon-first and prefers earlier hours on ties. Capacity is two VMs per group-hour, minus headroom reserved for forecast arrivals.
3. **Confidence gate.** If the best option is to wait, the job waits only if the predicted saving (g/kWh) exceeds the forecast's typical error at that horizon (test-set MAE). Otherwise it runs now.
4. **Commit and autoscale.** Jobs placed in the current hour are packed onto VMs (first-fit); the number of VMs per group follows from the plan.

5.2 Policies compared

Table 6. Policies evaluated on the same job stream.

Policy	Forecast used	Where it runs	Gate / buffer
FIFO (baseline)	none: run immediately	IN-WE only	– / –
FIFO in IN-SO (reference)	none: run immediately	IN-SO only	– / –
Simple carbon rule	persistence (same hour yesterday)	both groups	no / yes
CarbonShift	XGBoost carbon + workload	both groups	yes / yes
Oracle (upper bound)	actual future carbon and arrivals	both groups	– / no

5.3 VM controller: autoscaling on real VMs

On AWS the plan must become real VM actions. A t3.micro took about **25 s** from the start request until its worker was polling the queue, i.e. a large fraction of a simulated hour. The controller therefore (i) **starts VMs ahead of planned work**, looking ahead as many simulated hours as the boot time spans; (ii) applies a **cool-down** so a VM is stopped only after it has been unneeded for that long and its queue is empty (AWS cannot restart a VM that is still stopping, and bills at least 60 s per start); (iii) counts work still **in the queue** before sending more, so urgent jobs are never queued behind a backlog; and (iv) **always stops every VM** when the run ends or is interrupted.

5.4 Energy, carbon and cost model

Energy follows the Cloud Carbon Footprint linear model with AWS coefficients: per VM-hour, PUE · (vCPUs · P*idle* + *P*mem + (P*max* − *P*idle) · busy vCPU-hours), with P*idle* = *0.74 W, P*max = 3.5 W per vCPU, 0.392 W/GB of memory and PUE = 1.135. Carbon multiplies each group-hour's energy by the **actual** carbon intensity of that hour (never the forecast). Cost uses the AWS Price List API: t3.micro $0.0132/h plus $0.005/h for its public IPv4 address while running, and $0.768 per month for its 8 GB gp3 disk.

6. Implementation on AWS

All resources were created in the AWS console (and are reproducible with infra/create.sh), named carbonshift-* and tagged project=carbonshift, so that infra/teardown.sh deletes exactly these and nothing else in the shared account. Table 7 lists them; Figures 5–8 show them in the console and from the AWS CLI.

Table 7. AWS resources used by CarbonShift (region ap-southeast-2).

Resource	Configuration	Purpose
4 × EC2 t3.micro	Ubuntu 24.04 (Canonical), 8 GB gp3, IMDSv2 only, unlimited credits	Worker VMs, 2 per group
2 × SQS standard queue	visibility 120 s, long polling 20 s, retention 1 day	Job queue per group
DynamoDB table	carbonshift-results, on-demand, key job_id	Heartbeats and job results
IAM role + instance profile	SQS receive/delete on carbonshift-jobs-*, DynamoDB PutItem on the table, SSM	Least-privilege worker access
Security group	no inbound rules, outbound only	No SSH; nothing can connect in
AWS Budget	$10 monthly alert	Cost safety net

Worker. Each VM boots with a cloud-init script that installs boto3 and runs worker.py as a systemd service. The worker long-polls its group's queue, runs one job at a time (a CPU-bound Monte-Carlo estimate of π for cpu_seconds divided by the time-compression factor), writes the timings and result to DynamoDB, deletes the message and writes a heartbeat on start. A smoke test confirmed that a 300 CPU-second job ran for exactly 5.00 s at 60× compression on a VM of the correct group.

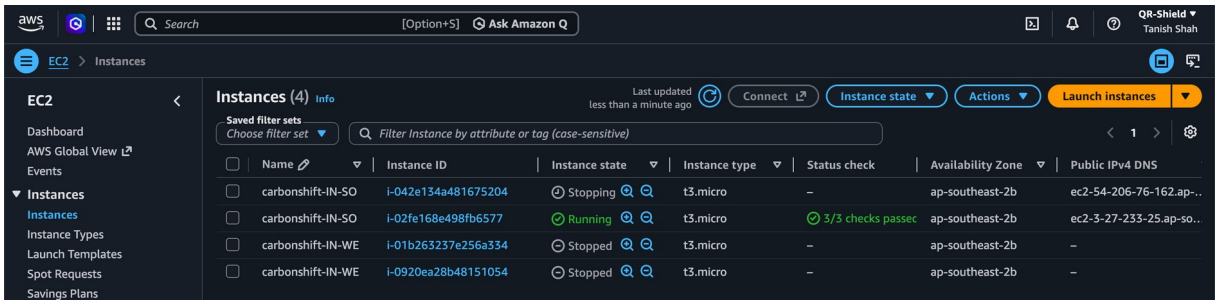


Figure 5. EC2 console: the four CarbonShift workers (two per group) as the controller switches them on and off.

```
AWS CLI (read-only) - CarbonShift resources in ap-southeast-2

$ aws sqs list-queues --queue-name-prefix carbonshift
https://sqs.ap-southeast-2.amazonaws.com/428521272060/carbonshift-jobs-IN-SO
https://sqs.ap-southeast-2.amazonaws.com/428521272060/carbonshift-jobs-IN-WE

$ aws sqs get-queue-attributes  (both queues)
carbonshift-jobs-IN-WE: {"VisibilityTimeout":"120","MessageRetentionPeriod":"86400","ReceiveMessageWaitTimeSeconds":"20"}
carbonshift-jobs-IN-SO: {"VisibilityTimeout":"120","MessageRetentionPeriod":"86400","ReceiveMessageWaitTimeSeconds":"20"}

$ aws dynamodb describe-table --table-name carbonshift-results
  name=carbonshift-results  status=ACTIVE  key=job_id  billing=PAY_PER_REQUEST (on-demand)

$ aws ec2 describe-security-groups --group-names carbonshift-worker-sg
  id=sg-00ffc90c10a2b9bf1  inbound_rules=0  outbound=0.0.0.0/0 (all)

$ aws iam list-attached-role-policies / get-role-policy  --role-name carbonshift-worker-role
  managed: AmazonSSMManagedInstanceCore
  inline: sqs:ReceiveMessage, sqs:DeleteMessage, sqs:ChangeMessageVisibility, sqs:GetQueueAttributes -> arn:aws:sqs:ap-southeast-2:428521272060:carbonshift-jobs-*
  inline: dynamodb:PutItem -> arn:aws:dynamodb:ap-southeast-2:428521272060:table/carbonshift-results
```

Figure 6. AWS CLI evidence of the queues, results table, security group (zero inbound rules) and least-privilege IAM role.

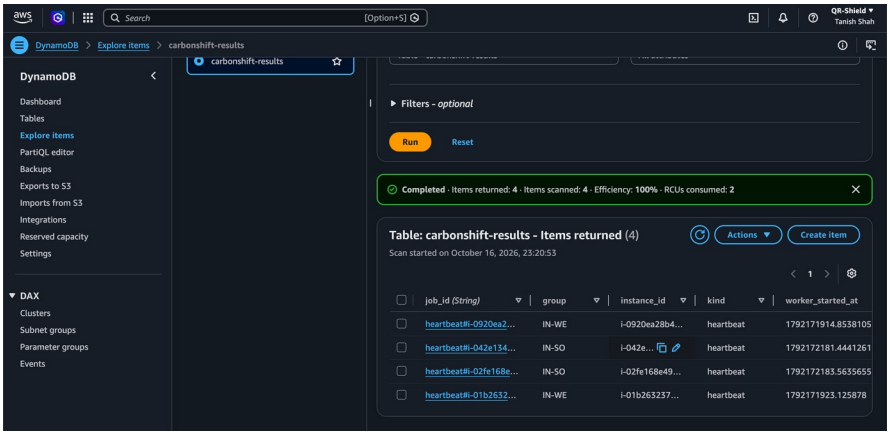


Figure 7. DynamoDB console: one heartbeat per VM, written by each worker when it starts.

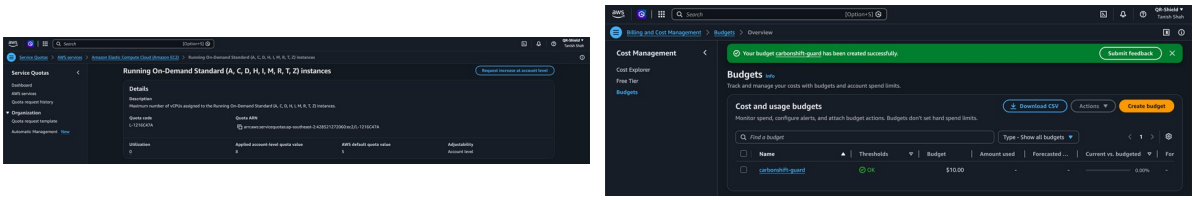


Figure 8. Left: the account's 8-vCPU on-demand quota, enough for 4 × t3.micro. Right: the $10 budget alert used as a cost guard.

6.1 Dashboard

A Streamlit dashboard follows a run live (refreshing every 3 s) or replays a finished one. It explains the current decision in one sentence, shows four key numbers, the carbon intensity so far together with the 24-hour forecast the scheduler is using, each VM with its instance ID and EC2 state, jobs per hour, VMs over time, and a timeline of every job.

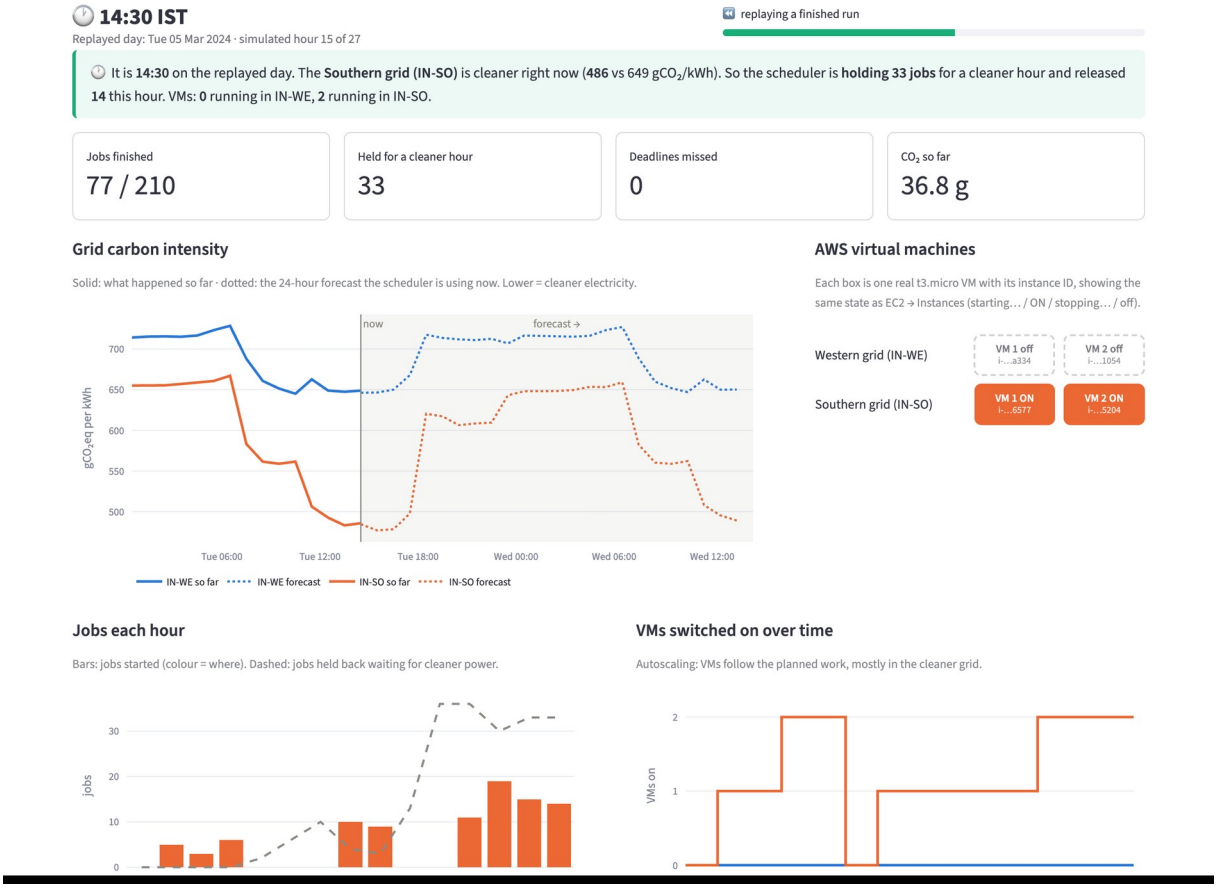


Figure 9. Dashboard during the CarbonShift run at 14:30 IST: jobs are being held for the cleaner afternoon, both IN-SO VMs are on and the IN-WE VMs are off.

7. Evaluation: full-year simulation

The simulator replays all of 2024 hour by hour (70,189 jobs, two groups of at most two VMs) with the same scheduler code as the live controller. Table 8 compares the policies at a 12-hour deadline slack.

Table 8. Full-year results at 12 h slack (prediction, utilisation, latency, energy, carbon and cost).

Policy	CO_2 (kg)	Saved vs FIFO	vs Oracle	Energy (kWh)	VM util.	Avg wait (h)	Missed	Cost ($)
FIFO, IN-WE (baseline)	34.3	0.0%	+46.8%	52.6	68%	1.0	0.00%	243
FIFO, IN-SO	27.9	18.8%	+19.2%	52.6	68%	1.0	0.00%	243
Simple carbon rule	23.7	30.9%	+1.5%	45.2	95%	5.4	0.00%	198
CarbonShift, no gate	23.6	31.2%	+1.0%	45.3	94%	5.1	0.00%	199
CarbonShift	23.6	31.1%	+1.1%	45.6	93%	4.2	0.00%	202
Oracle	23.4	31.9%	+0.0%	45.1	95%	5.5	0.00%	198

Carbon. CarbonShift emits 31% less CO_2 than FIFO and is within 1.1% of the oracle. Moving jobs to the cleaner Southern grid alone saves 19%; shifting them into the midday solar hours adds the rest. **Utilisation and energy.** Packing deferred jobs into fewer, fuller VM-hours raises utilisation from 68% to 93% and cuts energy by 13%. **Latency.** The price is an average wait of 4.2 h instead

of 1.0 h, still well inside the 12 h slack, with 0.00% misses. **Cost.** VM cost falls with the VM-hours; total cost is slightly higher only because four disks are provisioned instead of two.

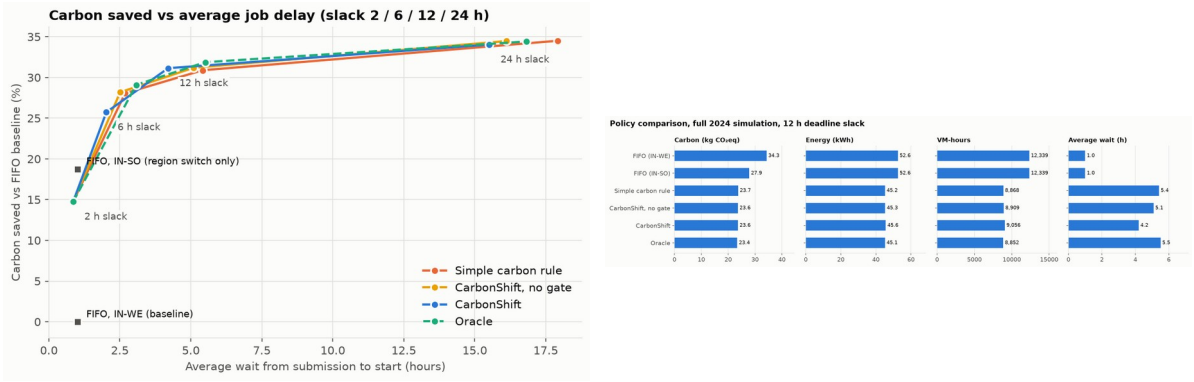


Figure 10. Left: carbon saved versus average delay for deadline slacks of 2, 6, 12 and 24 h. Right: carbon, energy, VM-hours and waiting time per policy at 12 h slack.

Table 9. CarbonShift across deadline slack values: more patience, more carbon saved.

Deadline slack	2 h	6 h	12 h	24 h
CO_2 saved vs FIFO	14.7%	25.7%	31.1%	34.0%
Average wait (h)	0.9	2.0	4.2	15.5
Deadline misses	0.01%	0.00%	0.00%	0.00%

With only 2 h of slack no job can wait for a cleaner hour, so all of the saving comes from choosing the cleaner group. A notable and honest finding is that the ML forecast is only slightly better than persistence *for scheduling*: the simple carbon rule reaches 30.9% against 31.1% for CarbonShift and 31.9% for the oracle. India's grid repeats almost the same daily shape, so even a naive forecast ranks the hours correctly; the ML model mainly helps on unusual days and with lower waiting time.

8. Evaluation: live runs on AWS

The controller replayed **Tuesday 5 March 2024** (an ordinary weekday; 210 jobs, 6 h slack) on the real VMs at 60× speed (one simulated hour per real minute), once with CarbonShift and once with FIFO, and replayed **Friday 8 March 2024 with a workload burst** (3× arrivals from 10:00 to 16:00) with CarbonShift. Every job travelled through SQS, ran on an EC2 worker and reported to DynamoDB.

Table 10. Measured results of the three final live runs on AWS.

Run (real AWS)	Jobs	Missed	Avg wait (h)	VM-hours	Util.	CO_2 (g)	Sim. (g)	AWS cost
CarbonShift, 5 Mar	210	0.0%	2.43	39	60%	88.2	76.1	$0.012
FIFO, 5 Mar	210	0.0%	0.91	46	51%	116.6	103.0	$0.014
CarbonShift, burst 8 Mar	372	0.0%	3.15	59	73%	160.2	142.8	$0.018

On identical jobs and conditions, CarbonShift used **24% less CO_2 than FIFO on real VMs** (88 g vs 117 g) with 0.0% missed deadlines, at the cost of a longer average wait (2.4 h vs 0.9 h). 100% of its jobs ran in the Southern group. In the burst run the IN-SO group became saturated and the scheduler used the **IN-WE VMs as overflow** (111 of 372 jobs, up to 2 IN-WE VMs on), keeping misses at 0.0%. Each run took about 25 real minutes and cost about one US cent.

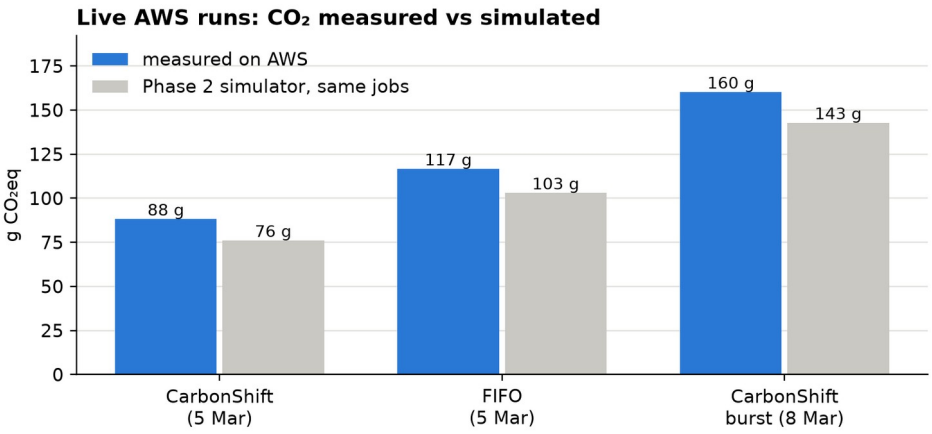


Figure 11. CO_2 of the live runs measured on AWS compared with the simulator's prediction for exactly the same jobs.

Real cloud vs simulation. Measured CO_2 is higher than the simulator predicts because real VMs must be started before work arrives and kept on briefly afterwards; an idle VM still draws about 2 W. The simulator ignores boot time and switches VMs on only in the hours they run jobs. This overhead affects every policy, which is why the fair comparison is CarbonShift against FIFO on the same real VMs, as in Table 10.

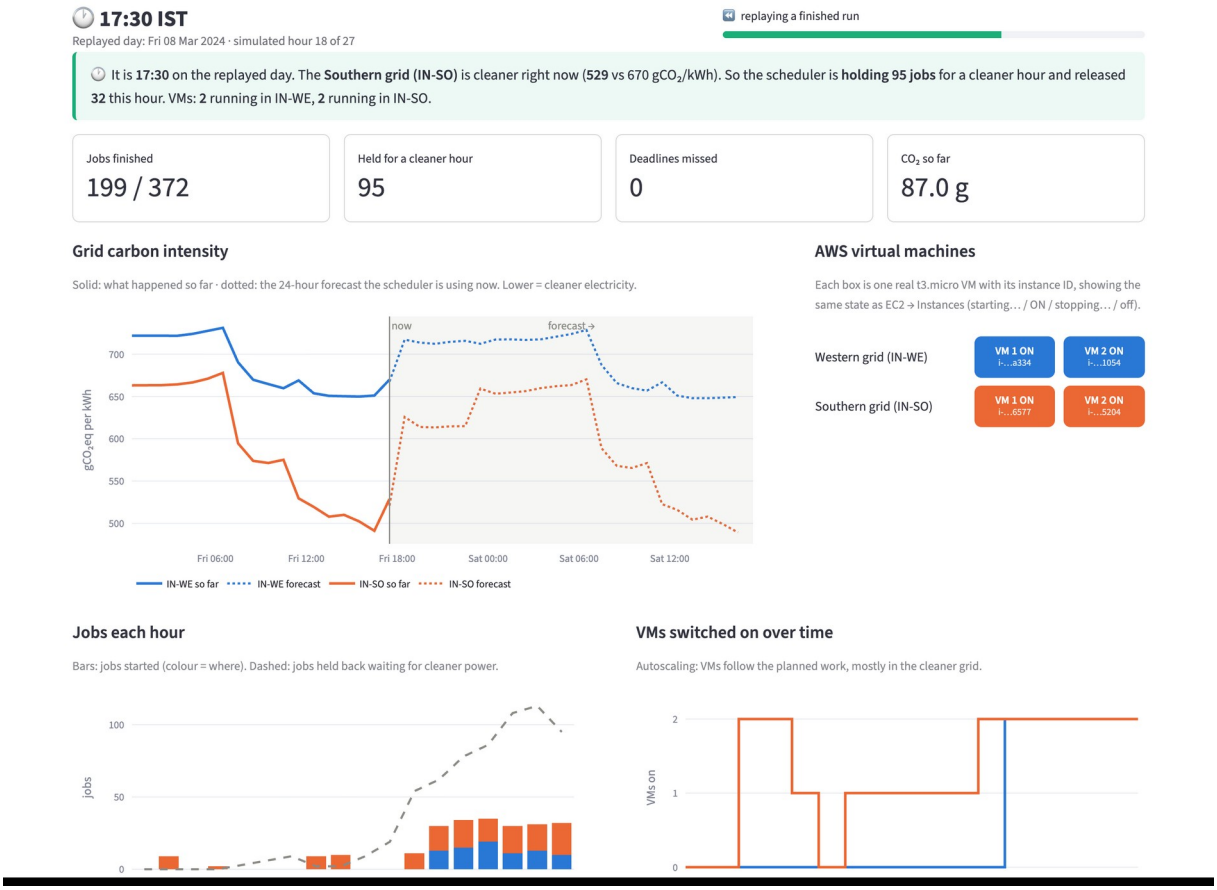


Figure 12. Dashboard during the burst run: the Southern group is full, so the Western-group VMs are switched on for overflow jobs.

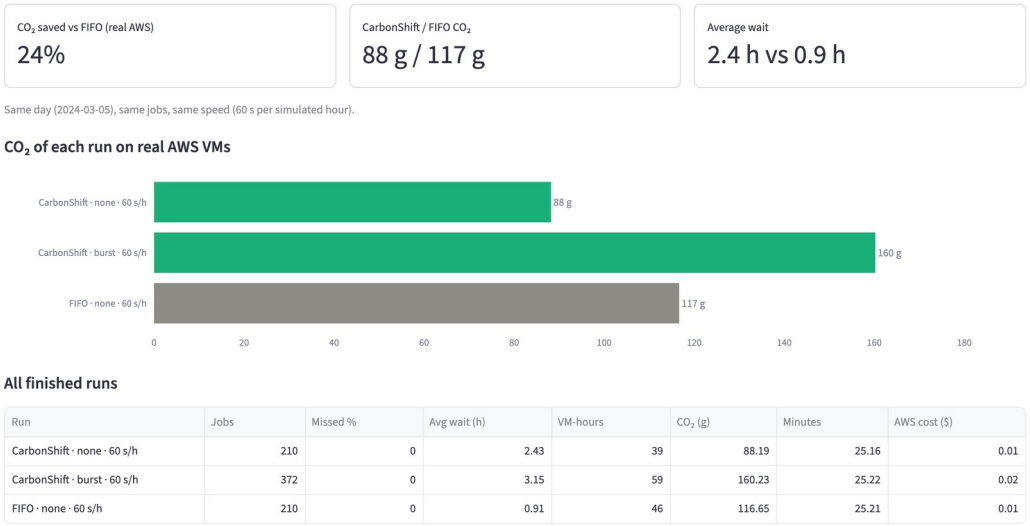


Figure 13. Dashboard results tab: the three live runs side by side.

```
$ python -m src.cloud.controller --policy carbonshift --yes

run cs-20261017-215010 | policy=carbonshift scenario=none | 210 jobs submitted 05 Mar 00:30 → 06 Mar 00:30 IST | slack 6 h | 1 sim hour = 60 s | groups IN-WE, IN-SO
...
[05 Mar 07:30 IST] sent 10 (IN-WE 0 IN-SO 10) | waiting   4 | VMs IN-WE 0→0 IN-SO 1→1 | done 14/210 | CO₂ 14.3 g | missed 0
[05 Mar 08:30 IST] sent  9 (IN-WE 0 IN-SO 9) | waiting   3 | VMs IN-WE 0→0 IN-SO 1→1 | done 23/210 | CO₂ 17.2 g | missed 0
[05 Mar 09:30 IST] sent  0 (IN-WE 0 IN-SO 0) | waiting  13 | VMs IN-WE 0→0 IN-SO 1→1 | done 33/210 | CO₂ 20.0 g | missed 0
[05 Mar 10:30 IST] sent  0 (IN-WE 0 IN-SO 0) | waiting  36 | VMs IN-WE 0→0 IN-SO 1→1 | done 33/210 | CO₂ 21.1 g | missed 0
  start IN-SO (i-042e134a481675204)
[05 Mar 11:30 IST] sent 11 (IN-WE 0 IN-SO 11) | waiting  36 | VMs IN-WE 0→0 IN-SO 1→2 | done 33/210 | CO₂ 23.3 g | missed 0
[05 Mar 12:30 IST] sent 19 (IN-WE 0 IN-SO 19) | waiting  30 | VMs IN-WE 0→0 IN-SO 2→2 | done 44/210 | CO₂ 27.0 g | missed 0
[05 Mar 13:30 IST] sent 15 (IN-WE 0 IN-SO 15) | waiting  33 | VMs IN-WE 0→0 IN-SO 2→2 | done 62/210 | CO₂ 32.0 g | missed 0
[05 Mar 14:30 IST] sent 14 (IN-WE 0 IN-SO 14) | waiting  33 | VMs IN-WE 0→0 IN-SO 2→2 | done 77/210 | CO₂ 36.8 g | missed 0
[05 Mar 15:30 IST] sent 16 (IN-WE 0 IN-SO 16) | waiting  30 | VMs IN-WE 0→0 IN-SO 2→2 | done 91/210 | CO₂ 41.1 g | missed 0
[05 Mar 16:30 IST] sent 16 (IN-WE 0 IN-SO 16) | waiting  30 | VMs IN-WE 0→0 IN-SO 2→2 | done 108/210 | CO₂ 46.2 g | missed 0
...
all 210 jobs finished
stopping VMs: i-02fe168e498fb6577
```

Figure 14. Controller output during the CarbonShift live run: per simulated hour, jobs released per group, jobs held, VM targets, progress and CO_2.

8.1 Engineering issues found on real hardware

The live runs exposed three problems that the simulator could not show. (i) **Stop/start thrash:** VMs stopped as soon as an hour had no work could not be restarted while still "stopping", delaying jobs by an hour; a cool-down fixed it and brought the average wait back to the simulator's value. (ii) **Queue backlog:** when a VM was still booting, jobs spilled into the next hour and an urgent job was queued behind them and missed its deadline by 41 minutes (0.5% misses in that run). Counting queued work before sending more fixed this. (iii) **Controller stall:** in one run, a single AWS API call from the laptop hung for about three minutes; the controller then replayed three simulated hours within seven seconds, leaving the VMs no time to finish and causing 3.3% misses. The controller now pauses the simulated clock when it falls behind instead of skipping hours, and that run was repeated (Table 10). These issues are typical of real autoscalers and are a useful lesson of deploying on a real cloud.

9. What happens when the prediction is wrong

We injected two kinds of forecast error into the full-year simulation, once a week each, and compared CarbonShift with and without its two safety mechanisms:

- **Surprise cloudy day:** actual carbon intensity is 25% higher than forecast from 08:00 to 18:00 IST in both grids; the forecasters only learn about it once it is observed.

- **Workload burst:** 3× the normal arrival rate for six hours from 10:00 IST, which the workload forecaster does not predict.

Table 11. Failure injection results (full-year simulation; "near event" = jobs whose deadline window overlaps an injected event).

Scenario (slack)	Policy	Extra CO_2 vs oracle	Missed (all jobs)	Missed (jobs near event)	Avg wait (h)
None (12 h)	CarbonShift	+1.1%	0.00%	–	4.2
Cloudy (12 h)	CarbonShift	+2.0%	0.00%	0.00%	4.5
Cloudy (12 h)	Adaptive gate	+4.9%	0.00%	0.00%	3.3
Burst (6 h)	FIFO	+34.1%	14.03%	40.9%	2.4
Burst (6 h)	**CarbonShift**	+4.1%	0.072%	0.092%	2.3
Burst (6 h)	No buffer	+2.2%	0.079%	0.158%	2.7
Burst (6 h)	No gate, no buffer	+0.1%	0.180%	0.354%	3.2

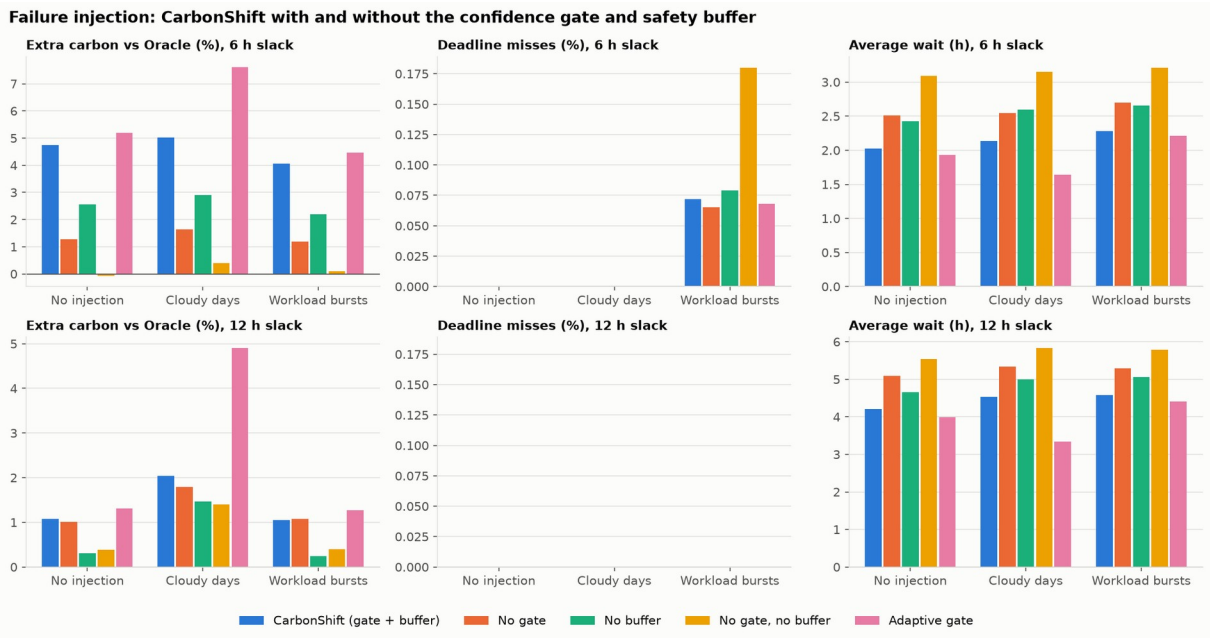


Figure 15. Failure injection: extra carbon versus the oracle, deadline misses and waiting time, with and without the confidence gate and safety buffer.

Wrong carbon forecast. On surprise cloudy days CarbonShift's gap to the oracle grows from 1.1% to 2.0%, but no deadline is missed: re-planning every hour uses the observed values as soon as they arrive, and the cost of the error is bounded because jobs deferred to a "clean" hour run there anyway. We also tested an **adaptive gate** that stops trusting forecasts after a large error. It backfired (+4.9%): "run now" during a cloudy midday meant running in the dirtiest hours. We therefore kept the simple gate, which trades a little carbon for lower waiting time.

Wrong workload forecast. During bursts FIFO, confined to one group, missed 41% of the affected jobs. CarbonShift spreads the load across both groups and over time and missed 0.09%; removing the **safety buffer** doubles the misses and removing both mechanisms quadruples them. The live burst run on AWS (Section 8) confirmed this behaviour on real VMs, with the Western-group VMs switched on as overflow capacity.

10. Discussion and limitations

- **Region choice dominates.** Because the Southern grid was cleaner in every hour of 2024, most of the saving comes from placing work there; time shifting adds the rest. In regions without such a gap, time shifting would matter more.
- **Data.** About 96% of the carbon values are Electricity Maps estimates, and the workload is synthetic (but seeded and realistic in shape). Results should be validated on measured data and real job traces.
- **Replay instead of live data.** The live demo replays 2024 at 60× speed so decisions can be scored against the recorded truth and policies compared on identical inputs. Connecting to a live carbon-intensity API is future work.
- **Single AWS region.** The free plan allows only ap-southeast-2, so both groups run in Sydney and are bound to Indian grid traces. This does not change the scheduling logic, but a production system would use real regions (e.g. ap-south-1 and ap-south-2).
- **Energy model.** The Cloud Carbon Footprint linear power model and hourly VM billing granularity are approximations; embodied emissions and network energy are not included.
- **Real-cloud overhead.** VM boot time and the cool-down add idle VM-hours that a simulation ignores; a pool of always-warm VMs or faster-starting containers would reduce it.

11. Conclusion and future work

CarbonShift shows that a small ML-based resource manager can cut the carbon footprint of deferrable cloud batch work substantially without breaking deadlines. XGBoost forecasts 1–24 hours ahead with 1.1–1.9% error; the rolling-horizon scheduler saves 31% CO_2 over a full simulated year while staying within 1.1% of an oracle; and on real AWS VMs it used 24% less CO_2 than FIFO for the same jobs. When forecasts are wrong, hourly re-planning, the confidence gate and the safety buffer keep the damage small.

Future work: live carbon-intensity data, real multi-region deployment, uncertainty-aware (quantile) forecasts in the gate, containers or a warm pool to cut boot overhead, and real job traces.

References

[1] Electricity Maps. Hourly carbon intensity datasets, zones IN-WE and IN-SO, 2021–2024. https://portal.electricitymaps.com

[2] Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of KDD '16, 785–794.

[3] Wiesner, P., Behnke, I., Scheinert, D., Gontarska, K. and Thamsen, L. (2021). Let's Wait Awhile: How Temporal Workload Shifting Can Reduce Carbon Emissions in the Cloud. Proceedings of Middleware '21.

[4] Radovanović, A. et al. (2023). Carbon-Aware Computing for Datacenters. IEEE Transactions on Power Systems, 38(2).

[5] Cloud Carbon Footprint. Methodology (energy coefficients and PUE for AWS). https://www.cloudcarbonfootprint.org/docs/methodology

[6] Amazon Web Services. Amazon EC2 User Guide; Amazon SQS Developer Guide; Amazon DynamoDB Developer Guide; AWS Price List API. https://docs.aws.amazon.com

Appendix A. How to reproduce

```
python -m src.data.load_carbon; python -m src.data.eda    # Phase 1: data, EDA
```

```
python -m src.models.train; python -m src.models.workload # forecasters
python -m src.sim.simulator                               # Phase 2: simulation
infra/create.sh --plan | infra/vms.sh status | infra/teardown.sh
python -m src.cloud.controller --policy carbonshift --yes # live AWS run
python -m src.cloud.controller --seconds-per-hour 20 --prewarm   # demo speed
streamlit run src/dashboard/app.py                        # dashboard
```

All tunables (regions, prices, energy coefficients, scheduler weights, time compression) are in config.yaml; random seeds are fixed. 14 automated tests cover feature leakage, simulator invariants (no job before submission, no VM above 100%, work conserved, deadlines met) and the controller end to end with fake VMs.